FIAP

**FASE 3** | A AURORA ESTABELECE OS PRIMEIROS SISTEMAS DA COLÔNIA

02

# SISTEMA DE GESTÃO INTELIGENTE DA COLÔNIA

Relatório técnico do SGIC para a missão Aurora Siger: organização dos dados, regras de decisão, previsão da energia eólica e análise do uso de energia.

GRUPO 2 · FELIPE DE BARROS FRANCO · GUILHERME SANTOS
CIÊNCIA DA COMPUTAÇÃO · FIAP · 2026

# VISÃO GERAL
# DO SISTEMA

**Grupo 2**

Felipe de Barros Franco · felipedebarrosfranco@gmail.com
Guilherme Santos · guilhermesantos3828@gmail.com

Código-fonte do protótipo: github.com/felipxv/FIAP-AuroraSiger_SGIC

O SGIC (Sistema de Gestão Inteligente da Colônia) é o sistema integrado pedido na atividade "A Aurora Estabelece os Primeiros Sistemas da Colônia". É um único programa em Python que organiza os dados da colônia, toma decisões por regras, prevê a geração eólica por regressão linear simples e compara geração com consumo. Ele usa só a biblioteca padrão, sem numpy, pandas ou sklearn, e fica dividido em funções. Dados, cálculos e decisões estão em arquivos separados, e o `main.py` só chama as funções e mostra os resultados.

TABELA 1 - ARQUIVOS DO PROTÓTIPO

| Arquivo | Papel | Funções |
|---|---|---|
| **dados_colonia.py** | dados | buscar_subsistema, listar_subsistemas |
| **regras_decisao.py** | decisões | regra_energia_baixa, regra_modo_economia, sistemas_a_desligar, decidir |
| **regressao.py** | lógica (previsão) | media, ajustar_reta, prever, r_quadrado |
| **analise_energia.py** | lógica (energia) | geracao_total, analisar_balanco, autonomia_horas |
| **main.py** | apresentação | mostrar_estrutura, mostrar_regras, mostrar_previsao, mostrar_analise_energia, mostrar_autonomia, main |

Os dados da colônia e o histórico da turbina são fictícios e foram criados para a demonstração. As regras de decisão trabalham com percentuais (nível de energia e consumo, de 0 a 100) e a análise de energia trabalha com kWh. O programa não converte uma escala na outra.

# ORGANIZAÇÃO DOS DADOS

Os dados ficam em estruturas chave-valor e listas. A colônia é um dicionário chamado `colonia`: cada chave é um sistema e o valor é outro dicionário com os subsistemas, então a estrutura forma uma árvore. O sistema energético, por exemplo, se divide em solar e eólico, e cada um guarda a geração em kWh e o status.

## CÓDIGO 1 - HIERARQUIA DO DICIONÁRIO COLONIA

TEXTO

```
colonia
├── energia
│   ├── nivel_percentual: 42      consumo_percentual: 68
│   ├── solar      {geracao_kwh: 42.0, status: operacional}
│   ├── eolica     {geracao_kwh: 18.5, status: operacional}
│   └── consumo_kwh: 70.0         reserva_kwh: 120.0
├── clima          {temperatura_c: -40.0, vento_kmh: 11.0, tempestade: False}
└── sistemas
    ├── suporte_vida, aquecimento, comunicacao        (essenciais)
    └── laboratorio, iluminacao_externa, estufa       (não essenciais)
```

Cada sistema tem três campos: se é essencial, se está ligado e quanto consome. O consumo dos seis sistemas soma 70 kWh, o mesmo valor de `consumo_kwh` da energia.

## TABELA 2 - SISTEMAS DA COLÔNIA

| Sistema | Essencial | Ligado | Consumo (kWh) |
|---|---|---|---|
| **suporte_vida** | sim | sim | 18,0 |
| **aquecimento** | sim | sim | 22,0 |
| **comunicacao** | sim | sim | 5,0 |
| **laboratorio** | não | sim | 9,0 |
| **iluminacao_externa** | não | sim | 6,0 |
| **estufa** | não | sim | 10,0 |
| **total** | | | 70,0 |

Duas funções recursivas navegam pela árvore. A `buscar_subsistema` recebe uma lista de chaves, uma por nível. A cada chamada ela tira a primeira chave, desce um nível e chama a si mesma com o resto da lista. Quando a lista acaba, devolve o que encontrou; se alguma chave não existe, devolve `None`. O caminho `["sistemas", "suporte_vida", "consumo_kwh"]` devolve 18.0 e `["energia", "nuclear"]` devolve `None`. A `listar_subsistemas` imprime a árvore inteira e acrescenta um recuo cada vez que encontra outro dicionário.

CÓDIGO 2 - NAVEGAÇÃO RECURSIVA EM DADOS_COLONIA.PY

PYTHON

```python
def buscar_subsistema(no, caminho):
    if len(caminho) == 0:
        return no
    chave = caminho[0]
    if not isinstance(no, dict) or chave not in no:
        return None
    return buscar_subsistema(no[chave], caminho[1:])
```

O histórico da turbina eólica fica em duas listas, `historico_vento` (km/h) e `historico_eolica` (kWh). A posição i das duas listas é a mesma medição. São os dados usados na regressão da seção 4.

# REGRAS DE DECISÃO

As regras ficam em `regras_decisao.py`. Os limites são constantes no começo do arquivo: `LIMITE_ENERGIA = 50` e `LIMITE_CONSUMO = 60`. Cada regra devolve uma mensagem quando a condição é verdadeira e `None` quando não se aplica.

TABELA 3 - REGRAS DE DECISÃO

| Regra | Condição | Saída |
|---|---|---|
| **simples** | energia < 50 | ALERTA: reduzir consumo |
| **combinada** | energia < 50 e consumo > 60 | ALERTA: ativar modo economia |
| **nenhuma** | energia ≥ 50 | NORMAL: operação dentro dos parâmetros |

A função `decidir` testa primeiro a regra combinada, que é a mais grave, e só chega na simples se a combinada não se aplicar. O enunciado usa como exemplo energia = 40 e consumo = 70, com a saída "ALERTA: reduzir consumo". No SGIC essa é a saída da regra simples, `regra_energia_baixa(40)`. Só que 70 também passa do limite de consumo alto (60), então a regra combinada vale e a decisão final para esse exemplo é "ALERTA: ativar modo economia". O estado atual da colônia (energia 42%, consumo 68%) cai no mesmo caso.

Quando há alerta, `sistemas_a_desligar` aplica a prioridade da colônia. Suporte de vida, aquecimento e comunicação são essenciais e nunca entram na lista. Os não essenciais que estão ligados são ordenados por bubble sort [F3], do que mais consome para o que menos consome. Desligando primeiro o que gasta mais, a colônia economiza o máximo desligando o menor número de sistemas. Com os dados atuais a ordem é estufa (10 kWh), laboratório (9 kWh) e iluminação externa (6 kWh), 25 kWh no total.

## TABELA 4 - CENÁRIOS DE TESTE EXECUTADOS PELO MAIN.PY

| Energia (%) | Consumo (%) | Decisão | Sistemas a desligar |
|---|---|---|---|
| 42 | 68 | ALERTA: ativar modo economia | estufa, laboratorio, iluminacao_externa |
| 40 | 70 | ALERTA: ativar modo economia | estufa, laboratorio, iluminacao_externa |
| 40 | 50 | ALERTA: reduzir consumo | estufa, laboratorio, iluminacao_externa |
| 80 | 70 | NORMAL: operação dentro dos parâmetros | nenhum |

A primeira linha é o estado atual da colônia; as outras são os cenários fixos da parte 2 do main.py.

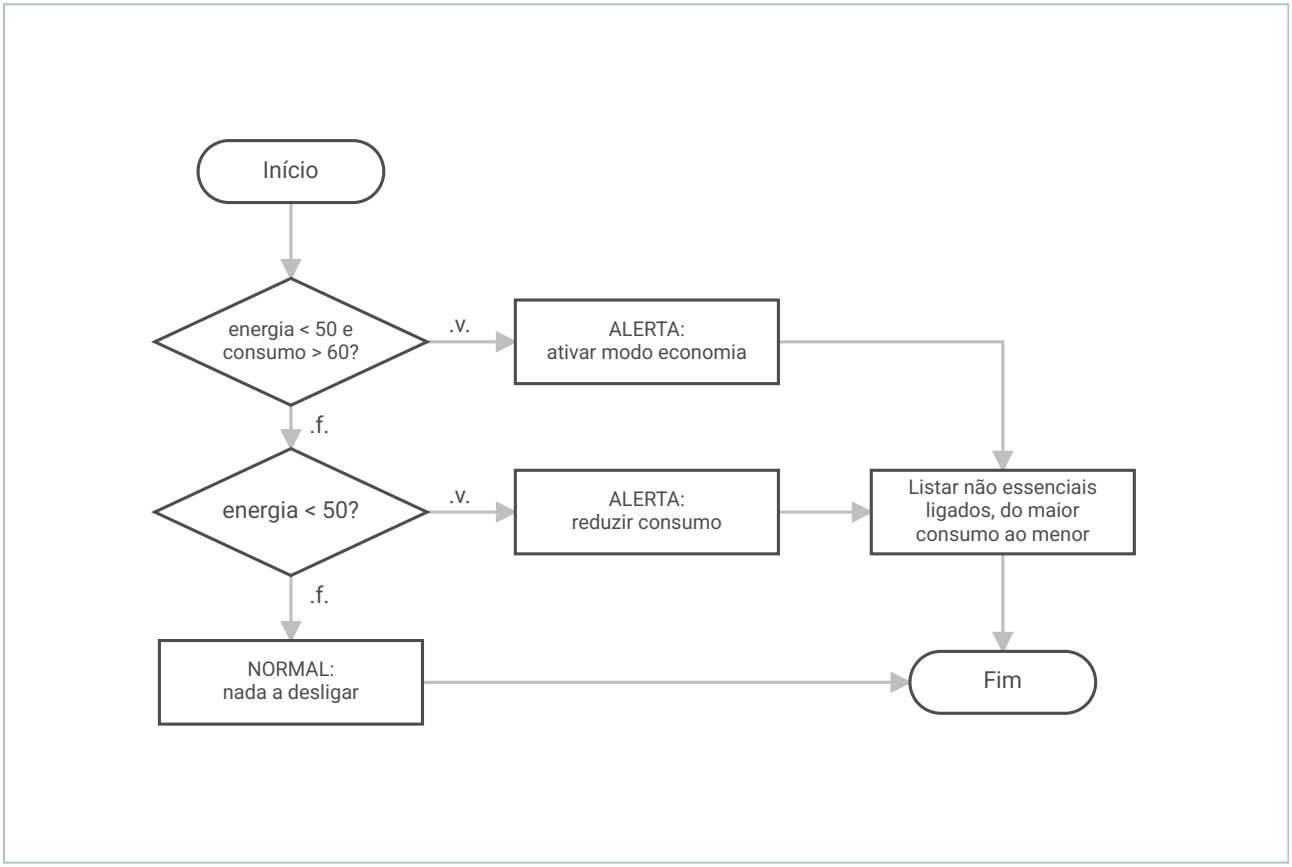



Figura 1 - Fluxo da função decidir
Fonte: Elaborado pelos autores

# MODELO
# DE PREVISÃO

Para prever a geração da turbina usamos regressão linear simples pelo método dos mínimos quadrados, como no Cap. 7 [F1]. O modelo é uma reta, energia = b0 + b1 · vento. Os coeficientes são calculados à mão em `ajustar_reta`, com laços e somas:

$$b1 = soma[(x - média_x) \cdot (y - média_y)] \,/\, soma[(x - média_x)^2]$$

$$b0 = média_y - b1 \cdot média_x$$

b1 é a inclinação, quanto a energia sobe para cada km/h a mais de vento. b0 é o intercepto. Com a reta pronta, `prever` calcula a energia para um vento que não está no histórico, e `r_quadrado` mede quanto a reta explica dos dados (1 é um ajuste perfeito).

## EXEMPLO DO ENUNCIADO

Com vento = [8, 10, 12] e energia = [20, 25, 30], as médias são 10 e 25. Os desvios do vento são −2, 0 e 2 e os da energia são −5, 0 e 5. A soma dos produtos dá 20 e a soma dos quadrados do vento dá 8, então b1 = 20 / 8 = 2,5 e b0 = 25 − 2,5 × 10 = 0. Para vento 11 a previsão é 2,5 × 11 = 27,5, o valor de aproximadamente 27 que o enunciado indica. O programa chega ao mesmo resultado.

## HISTÓRICO DA TURBINA

O histórico tem oito medições: `historico_vento = [6, 8, 9, 10, 12, 14, 15, 18]` (km/h) e `historico_eolica = [11.5, 15.0, 17.0, 19.5, 23.0, 26.5, 28.0, 33.0]` (kWh).

Com as oito medições a reta fica energia = 0,87 + 1,81 · vento, com R² = 0,997. Cada km/h a mais de vento rende cerca de 1,81 kWh, e a maior diferença entre o medido e a reta é de 0,53 kWh. Para o vento atual da colônia, 11 km/h, a previsão é 20,8 kWh. A geração eólica registrada nos dados é 18,5 kWh. Como os dados são fictícios e a reta é uma aproximação, os dois valores não precisam coincidir.

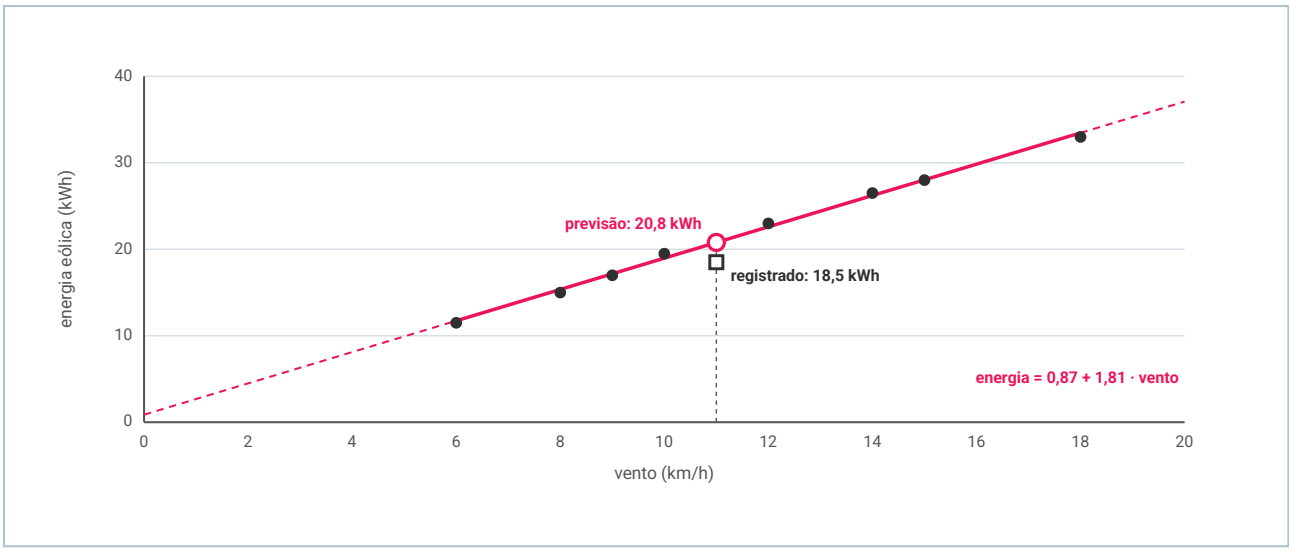



Figura 2 - Medições da turbina, reta ajustada e previsão para o vento atual
Fonte: Elaborado pelos autores

A reta tem limite. A potência que o vento entrega cresce com o cubo da velocidade [F2]: dobrar o vento multiplica a potência por oito. Na faixa medida, de 6 a 18 km/h, a reta acompanha bem os pontos, mas fora dela o erro cresce. Com vento zero ela daria 0,87 kWh, e uma turbina parada não gera nada. Por isso a reta aparece tracejada fora da faixa no gráfico e só deve ser usada para ventos parecidos com os do histórico.

# COMO O SGIC MELHORA O USO DE ENERGIA

A análise de energia trabalha em kWh. `geracao_total` soma a geração solar e a eólica e `analisar_balanco` compara o total com o consumo. Se o consumo passa da geração, há risco de faltar energia e a saída é "ALERTA: consumo maior que geração". Se a geração passa do consumo, a sobra seria desperdiçada e a saída é "SUGESTÃO: armazenar energia excedente". Com os dois iguais, a saída é "NORMAL: geração e consumo equilibrados". O `main.py` roda os dois exemplos do enunciado (geração 40 com consumo 70 e geração 80 com consumo 30) e chega às saídas pedidas.

Quando falta energia, a diferença sai da reserva, e `autonomia_horas` calcula por quantas horas ela aguenta esse déficit. Se não há déficit a função devolve `None`, porque a reserva não está sendo gasta.

$$autonomia = reserva \ / \ (consumo - geração)$$

Hoje a colônia gera 42,0 kWh solares e 18,5 kWh eólicos, 60,5 kWh no total, e consome 70,0 kWh. O resultado é o alerta de consumo maior que a geração, e o déficit de 9,5 kWh esvazia a reserva de 120 kWh em 12,6 horas. A previsão da seção 4 entra aqui: o `main.py` repete a análise trocando a geração eólica registrada pela prevista para o vento atual. A geração sobe para 62,8 kWh e a autonomia passa para 16,6 horas, mas o alerta continua, então a colônia sabe com antecedência que o vento esperado não cobre o consumo.

TABELA 5 - RESULTADO DA ANÁLISE DE ENERGIA

| Situação | Geração (kWh) | Consumo (kWh) | Saída | Autonomia |
|---|---|---|---|---|
| **exemplo do enunciado** | 40,0 | 70,0 | ALERTA: consumo maior que geração | - |
| **exemplo do enunciado** | 80,0 | 30,0 | SUGESTÃO: armazenar energia excedente | - |
| **colônia agora** | 60,5 | 70,0 | ALERTA: consumo maior que geração | 12,6 h |
| **com a eólica prevista** | 62,8 | 70,0 | ALERTA: consumo maior que geração | 16,6 h |

As regras completam a análise. Com o estado atual elas pedem modo economia e indicam estufa, laboratório e iluminação externa para desligar, 25 kWh no total. Se esses três forem desligados, o consumo cai de 70,0 para 45,0 kWh, abaixo dos 60,5 kWh gerados, e a análise de balanço passa a dar "SUGESTÃO: armazenar energia excedente": sobram 15,5 kWh para a reserva. Os 45 kWh restantes são exatamente o consumo dos três sistemas essenciais, que continuam ligados. O programa não faz essa simulação sozinho: a conta junta a lista de sistemas a desligar com `analisar_balanco` aplicada ao consumo que sobra.

Na prática, o SGIC ajuda de quatro formas:

- detecta energia baixa e consumo alto e escolhe a ação mais grave que se aplica;
- diz o que desligar sem tocar no suporte de vida, no aquecimento e na comunicação, começando pelo que mais consome;
- prevê a geração eólica pelo vento esperado, o que permite planejar o consumo antes de faltar energia;
- mostra quanto tempo a reserva aguenta e avisa quando há sobra para guardar.

# PREMISSAS E LIMITAÇÕES

- Os dados da colônia e o histórico da turbina são fictícios, criados para a demonstração.
- Energia e consumo das regras são percentuais; geração, consumo e reserva da análise de energia são kWh. As duas escalas não são convertidas uma na outra.
- No exemplo do enunciado (energia 40, consumo 70) a regra simples dá "ALERTA: reduzir consumo", mas a decisão final é o modo economia, porque a regra combinada é testada primeiro.
- A potência real do vento cresce com o cubo da velocidade [F2]. A reta é uma aproximação que vale na faixa de 6 a 18 km/h do histórico.
- A autonomia supõe que geração e consumo ficam constantes enquanto a reserva é usada.

# REPOSITÓRIO DO SISTEMA

O sistema está no repositório público github.com/felipxv/FIAP-AuroraSiger_SGIC. Ele roda com Python 3.12 ou mais recente, sem dependências: `python main.py` executa as quatro partes em sequência. O `README.md` explica o funcionamento, mostra um exemplo de entrada e saída e traz comandos para testar cada função isolada.

# REFERÊNCIAS

[F1] FIAP. Ciência da Computação, Fase 3, Cap. 7: regressão linear simples pelo método dos mínimos quadrados. São Paulo: FIAP.

[F2] FIAP. Ciência da Computação, Fase 3, Cap. 8: potência do vento e geração eólica. São Paulo: FIAP.

[F3] FIAP. *Os algoritmos que selecionam caminhos e ordenam prioridades de voo*. Fase 2, Cap. 5. São Paulo: FIAP, 2025.

Repositório do projeto: github.com/felipxv/FIAP-AuroraSiger_SGIC.